Journal of Information and Intelligence xxx (xxxx) xxx

Contents lists available at ScienceDirect

Journal of Information and Intelligence

journal homepage: www.keaipublishing.com/en/journals/journal-of-information-and-intelligence

AC-LSTM: Adaptive clockwork LSTM for network traffic prediction

Yunfeng Ge [a], Zonghuan Guo [a], Peng Wang [b], Xiaogang Li [a], Hongyan Li [a,*]

[a] State Key Laboratory of Integrated Service Networks, Xidian University, Xi'an 710071, China
[b] Pillar of Information Systems Technology and Design, Singapore University of Technology and Design, Singapore City 487372, Singapore

ARTICLE INFO

Keywords:
Network traffic forecasting
Machine learning
Network management
Traffic modeling

ABSTRACT

Accurate traffic load prediction on network links is essential for operators to optimize routing strategies based on predictive data. Deep time series analysis offers promising approaches to modeling network traffic flows. The traditional long short-term memory (LSTM) model is commonly used to capture complex nonlinear relationships in temporal data. However, LSTMs often face challenges with long-range forecasting, either leading to diminished accuracy due to vanishing gradients or incurring high computational costs. To address these issues, this paper introduces the adaptive clockwork LSTM, a modified LSTM-based model that aims to provide accurate long-term predictions while reducing computational complexity. Dividing the LSTM unit into multiple segments that operate at adaptively varying intervals, an adaptive correlation mechanism is used for efficient sampling. The experimental results show that it outperforms existing methods, achieving a 20% improvement in forecast accuracy and a 65% reduction in computational time. These improvements make it a promising solution to reduce the costs associated with network management.

1. Introduction

Network traffic workload refers to the amount of network flows that must be timely transmit to facilitate demand [1], which is crucial for maintaining quality of service (QoS). Network operators widely use a prediction-based closed-looploop network management framework to ensure QoS, including network slices, network function virtualization (NFV) and multi-access edge computing (MEC) [2,3]. Within these frameworks, advanced data analysis techniques, often leveraging deep learning and sophisticated data mining, are increasingly applied not only for workload pediction but also for other critical tasks such as fine-grained traffic analysis [4], identifying malicious social bot traffic [5], and detecting distributed denial-of-service (DDoS) attacks [6]. Eventually, selecting appropriate prediction network is pivotal for network management. However, predicting network traffic workload is challenging due to the differentiate temporal dependencies and heterogeneity of service flows, such as video streaming, web browsing, and file transfer services. This resulting dynamic, time-varying characteristics of network link workloads, presenting significant challenges for traditional network management frameworks.

Leveraging deep learning (DL) algorithms in time series domain is widely used for network traffic prediction [7–14]. We regard network traffic load prediction as a multi-step time series forecasting problem. The potential swift inference and accurate prediction makes deep time series analysis a promising technology to address the challenges [15,16,17]. Within the AI-driven time series forecasting, two primary model architectures are widely used: recurrent neural networks (RNNs) and transformers. RNNs are popular for capturing temporal dependencies in time series data but suffer from vanishing or exploding gradient issues, which hinder their ability

* Corresponding author.
E-mail addresses: 22011110189@stu.xidian.edu.cn (Y. Ge), guozonghuan@xidian.edu.cn (Z. Guo), peng_wang2@sutd.edu.sg (P. Wang), xgli_1@stu.xidian.edu.cn (X. Li), hyli@xidian.edu.cn (H. Li).

https://doi.org/10.1016/j.jiixd.2025.06.004
Received 12 April 2025; Received in revised form 16 June 2025; Accepted 18 June 2025
Available online xxxx
2949-7159/© 2025 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Please cite this article as: Y. Ge et al., AC-LSTM: Adaptive clockwork LSTM for network traffic prediction, Journal of Information and Intelligence, https://doi.org/10.1016/j.jiixd.2025.06.004

to accurately model long-term dependencies. Although long short-term memory (LSTM) networks [18] and their subsequent research [7,8,10,13] focus on improving prediction performance, they fall in limit progress because their redundant aggregation caused exponential growth parameters and hinder their ability to model long-term dependencies. Transformer-based models [19,20,21,22] have gained significant attention in recent years for their ability to model temporal data, particularly through point-wise attention mechanisms. However, their squared computation complexity presents challenges efficient network load prediction. Although recent innovations, such as the log-sparse mechanism [19,20], patching mechanism [21], and auto-correlation mechanism [22], try to tackle this problem, they struggle to achieve swift feature modeling due to inherent transformer architecture.

To achieve effective and efficient feature extraction, this paper proposes adaptive clockwork long short-term memory (AC-LSTM). First, drawing inspiration from the sampling theorem in signal analysis, we divide the hidden layer units in LSTM neurons into fixed sub-units, each responsible for extracting information at different sampling intervals. This helps to retain both long-range and short-range features. Second, to determine an appropriate sampling interval for each sub-unit, we propose an adaptive interval sampling scheme, based on the auto-correlation mechanism from stochastic process theory. Finally, the training pipeline for sub-units state update is provided. AC-LSTM enable the model to extract information more effectively and efficiently. We evaluate the performance of AC-LSTM in predicting traffic volume, compared with other existing methods. The results demonstrate significant improvements: AC-LSTM outperforms the vanilla LSTM by 23% on average. When compared to transformer-based models, AC-LSTM achieves a 65% reduction in running time and a 20% improvement in precision, as depicted in Fig. 1. In summary, the major contributions are as follows:

- We propose AC-LSTM, an adaptive clockwork long short-term memory model. It integrates an adaptive clockwork mechanism, inspired by sampling theory, to efficiently capture both short-term and long-term dependencies in network traffic load.
- We design an adaptive auto-correlation module that dynamically determines the optimal sampling intervals within the clockwork mechanism, allowing the model to adaptively process temporal correlations.
- Through comprehensive experiments on real-world datasets, we demonstrate that AC-LSTM achieves state-of-the-art performance, significantly outperforming existing LSTM and transformer models in both prediction precision and computational efficiency.

The paper is organized as follows. Section 2 presented the network prediction problem description in network management system. Section 3 summarized the related work on network traffic prediction and deep time series prediction. System model is described in Section 4. The detailed experiments are demonstrated in Section 5. Conclusion is discussed in Section 6.

2. Network prediction problem description

Prediction models are essential in network management, enabling better resource allocation and routing strategies. A closed-loop control system can incorporate prediction models to manage network loads and update routing strategies, as shown in Fig. 2. For example, prediction-based dynamic routing algorithms optimize QoS criteria, which are essential for maintaining service level agreements [23,24]. However, existing models often involve redundant parameters and high space complexity, which hinders their practical application [25,26,27]. To address this, lightweight prediction models that balance speed and accuracy are necessary.

Long-term time-series forecasting facilitates long-range predictions, improving implementation efficiency. Under the rolling forecasting setting with a fixed size window, we have the input $\mathcal{X}^t = \left\{\mathbf{x}_1^t, \cdots, \mathbf{x}_{L_x}^t \middle| \mathbf{x}_i^t \in \mathbb{R}^{d_x}\right\}$ at time t, and the output is to predict the corresponding sequence $\mathcal{Y}^t = \left\{\mathbf{y}_1^t, \cdots, \mathbf{y}_{L_y}^t \middle| \mathbf{y}_i^t \in \mathbb{R}^{d_y}\right\}$. Long-range time-series forecasting encourages a longer output length L_y than previous works [28] and the feature dimension is not limited to univariate case ($d_y \geqslant 1$).

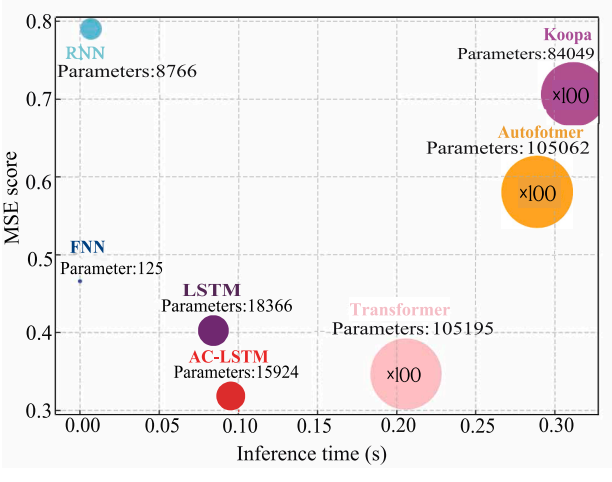


Fig. 1. Model efficiency.

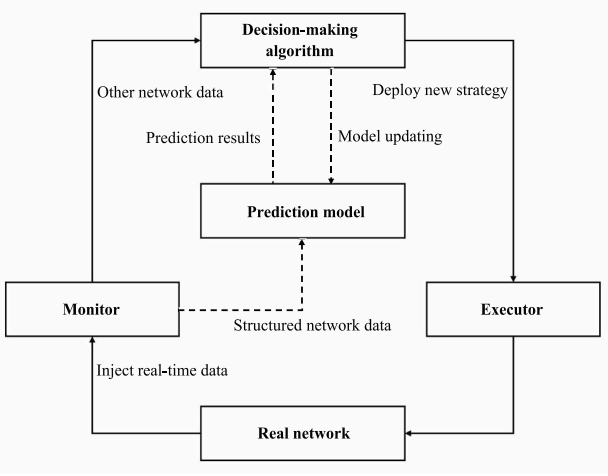


Fig. 2. The closed-loop control framework.

3. Related works

Network trafficprediction using deep learning approach. Network traffic forecasting, the task of predicting future traffic based on past values, is essential for effective network management. [29] proposes a zero-touch control mechanism for network slicing that forecasts capacity requirements. [30] employs logistic regression (LR) and support vector machine (SVM) to predict customer delay. In the domain of deep learning, [31] develops a smoothed long short-term memory (SLSTM) model for accurate traffic prediction in 5G networks, while [32] utilizes recurrent neural networks (RNNs) to forecast traffic capacity. A survey by Ref. [33] highlights the applications of machine learning techniques, including CNNs, LSTMs, radial basis function (RBF), eXtreme Gradient Boosting (XGBoost), and SVM in network management.

Table 1 presents a comprehensive overview of contemporary approaches using deep learning for network modeling. Numerous studies have aimed to predict network performance. LSTM models are widely used in machine learning to capture temporal representations in networks; however, they face challenges such as vanishing gradients and high computational complexity. Consequently, researchers have attempted to address these issues. As detailed below, our approach is primarily distinguished by its faster training, higher precision, use of a simple yet effective model, and reliance on publicly accessible datasets.

Time series prediction using deep learning model. Deep learning models have been proposed to tackle the time series prediction, with LSTM-based approaches emerging as a promising solution for predicting time series patterns. For example, [8] improves upon LSTM-based models by integrating an attention mechanism that connects long-sequence traffic values to current time steps, thereby improving prediction accuracy. [13] introduces multiple time-scale features to enhance long-term dependency modeling, while [35] combines LSTMs with deep neural networks (DNNs), incorporating autocorrelation coefficients to improve prediction accuracy. While these models effectively model dependencies, they often introduce redundant parameters that increase computational costs. The most related work is clockwork RNN [37]. In this model, the RNN's hidden state is partitioned into multiple sub-cells, and the model was sampled at predefined sampling intervals, specifically using powers of 2. In contrast to this approach, our work is based on the LSTM architecture, which is known for its superior performance over traditional RNNs. Considering that LSTMs maintain both a hidden state and a cell state, directly designing separate sub-cells for each would significantly increase model complexity and compromise interpretability. Therefore, inspired by the skip connection mechanism in ResNet, we propose to retain data useful for prediction by triggering these connections at specific intervals. In addition, we design an adaptive module to find the appropriate intervals.

Table 1
Tables of a comprehensive overview of the contemporary approaches.

Application area	Prediction model	Less training cost	Datasets
Wide area network (WAN) [8]	CRNN	Yes	Public
Traffic matrix (TM) prediction [13]	pLSTM	No	Public
Traffic matrix [34]	LSTM	Yes	Public
Network traffic prediction [35]	LSTM	No	Public
Wireless network [30]	SVM	Yes	Public
5G network traffic [31]	sLSTM	No	Public
Zero-touch network slicing [32]	3D-CNN	Yes	Private
Web traffic forecasting [12]	ARIMA, LSTM	No	Private
IoT traffic prediction	GRU-NN	Yes	Public
Real-time load prediction [36]	LSTM, GRU	No	Public
Traffic matrix (TM) prediction [7]	LSTM	No	Public
Network load prediction (This work)	AC-LSTM	Yes	Public

In recent years, transformer-based approaches have made significant strides in time-series prediction. Transformer [38,39], informer [20], and autoformer [22] have gained attention for their ability to handle temporal sequence forecasting. A key innovation in these models is the LogSparse mechanism, introduced in [20], which selects exponentially growing time intervals to reduce computational complexity. The auto-correlation mechanism in [22] further enhances these models by identifying period-based dependencies in time-series data. Despite their promising results, these models often excel in specific domains but struggle to achieve both high accuracy and efficiency in dynamic network environments.

4. System model

This section first reviews the basic LSTM unit. It then proposes a clockwork mechanism for LSTM along with an autocorrelation module. Finally, it presents a training algorithm to illustrate the complete training process.

4.1. Long short-term memory unit

The long short-term memory unit is widely adopted for processing sequential data due to its significant advantages over traditional recurrent neural networks (RNNs). The architecture of an LSTM, illustrated in Fig. 3(a), features a chain-like structure and sophisticated gating mechanisms (input, forget, and output gates). These gates regulate the flow of information, allowing the model to effectively capture long-term dependencies. This critically addresses the vanishing and exploding gradient problems that typically hinder traditional RNNs.

The internal state of each LSTM neuron is essential to its function. The cell state from the previous time step is denoted by $\mathbf{c}_{t-1}$, the prior hidden state is represented by $\mathbf{h}_{t-1}$, and $\mathbf{x}_t$ corresponds to the input vector at the current time step. The sigmoid function, denoted by σ, is commonly used as the activation function in the gating mechanism. The weight matrix is represented by $\mathbf{W}$, and the bias term is denoted by $\mathbf{b}$. The computations for the three gates can be derived from Fig. 3(a) as follows:

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_\mathbf{f} \cdot \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} + \mathbf{b}_\mathbf{f}\right), \tag{1}$$

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_\mathbf{i} \cdot \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} + \mathbf{b}_\mathbf{i}\right), \tag{2}$$

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_\mathbf{o} \cdot \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} + \mathbf{b}_\mathbf{o}\right), \tag{3}$$

$$\tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_\mathbf{c} \cdot \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} + \mathbf{b}_\mathbf{c}\right), \tag{4}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{5}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \tag{6}$$

where the intermediate state of the LSTM neuron is represented by the symbol $\tilde{\mathbf{c}}_t$, the symbol "tanh" represents the non-linear activation function, $\mathbf{c}_t$ symbolizes the storage unit of the neuron and the symbol $\odot$ symbolizes the dot product operation between two vectors.

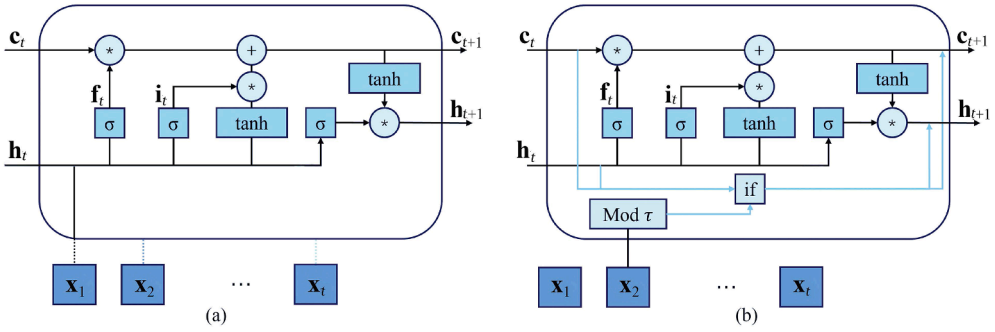


Fig. 3. Architecture comparison. (a) LSTM architecture. (b) AC-LSTM architecture.

4.2. Clockwork mechnism

Building upon the strengths of the LSTM model in handling temporal data and mitigating vanishing gradients, we propose an enhancement to specifically improve the capture of both long and short-range dependencies. This is achieved through the incorporation of a novel clockwork mechanism, drawing inspiration from the concept of downsampling. This mechanism is designed to process information at multiple time scales within the network by allowing different parts of the model to update at varying sampling interval.

The mechanism begins by dividing the hidden state and memory state units into k groups, where each group represents a specific range of sequence information. These groups are referred to as LSTM subunits.

As illustrated in Fig. 3(b), each LSTM subunit can skip irrelevant inputs based on a predefined interval, τ. It then determines whether to update or copy the hidden state and memory state depending on the modulus of the input sequence index and the preset value.

The LSTM has two state variables: the cell state $\mathbf{c}_t$ and the hidden state $\mathbf{h}_t$. Additionally, it incorporates three gates: the forget gate$\mathbf{f}_t$, output gate $\mathbf{o}_t$, and input gate $\mathbf{i}_t$. The clockwork frequency is denoted by τ_m. The cell state $\mathbf{c}_t$ and hidden state $\mathbf{h}_t$ are divided into multiple segments according to different sampling frequencies, represented as $\mathbf{c}_t^k$ and $\mathbf{h}_t^k$. The calculations for the clockwork LSTM layer are as follows:

$$\widehat{\mathbf{c}}_t^k = \mathrm{g}\left(\mathbf{W}_{\mathbf{xc}} \cdot \mathbf{x}_t + \mathbf{W}_{\mathbf{hc}} \cdot \mathbf{h}_{t-1}^k\right), \tag{7}$$

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_{\mathbf{xi}} \cdot \mathbf{x}_t + \mathbf{W}_{\mathbf{hi}} \cdot \mathbf{h}_{t-1}^k\right), \tag{8}$$

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_{\mathbf{xf}} \cdot \mathbf{x}_t + \mathbf{W}_{\mathbf{hf}} \cdot \mathbf{h}_{t-1}^k\right), \tag{9}$$

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_{\mathbf{xo}} \cdot \mathbf{x}_t + \mathbf{W}_{\mathbf{ho}} \cdot \mathbf{h}_{t-1}^k\right), \tag{10}$$

$$\mathbf{h}_t^k = \begin{cases} \mathbf{o}_t \odot \mathbf{f}\left(\mathbf{c}_{t-1}^k\right) & \text{if } t \bmod \tau_m \neq 0 \\ \mathbf{h}_{t-1}^k & \text{otherwise} \end{cases}, \tag{11}$$

$$\mathbf{c}_t^k = \begin{cases} \mathbf{f}_t \odot \mathbf{c}_{t-1}^k + \mathbf{i}_t \odot \widehat{\mathbf{c}}_t^k & \text{if } t \bmod \tau_m \neq 0 \\ \mathbf{c}_{t-1}^k & \text{otherwise} \end{cases}. \tag{12}$$

4.3. Adaptive auto-correlation module

Our method extends the capabilities of clockwork mechanisms for capturing multi-scale temporal dependencies. Specifically, we replace the manual, hyperparameter-based specification of sampling intervals with an adaptive determination process.

To adaptively learn the appropriate sampling intervals, we employ the concept of rolling auto-correlation, which identifies both long and short-range dependencies according to the following function:

$$R_\tau = \frac{1}{T}\left[\sum_{t=1}^{T-\tau}(X_t - \overline{X})(X_{t+\tau} - \overline{X}) + \sum_{t=1}^{\tau}(X_{T-\tau+t} - \overline{X})(X_t - \overline{X})\right], \tag{13}$$

where X represents the input sample data, $\overline{X}$ denotes the mean value of the sample data, and T represents the length of the sample data. R_τ is the rolling autocorrelation coefficient with an offset of τ.

Fig. 4 illustrates the structure of the AC-LSTM unit, where the state is partitioned and updated at varying time intervals based on long-and short-term dependencies. The varying time interval is not dependent on a set of hyper-parameters predefined by human expert. An appropriate sampling interval will be determined in a adaptive manner. Specifically, time series data is fed into the AC-LSTM model. We calculate the k lags exhibiting the highest autocorrelation using a formula. These identified lags are then utilized as the sampling interval, which is adaptively determined by time series characters. The rationale is that a high autocorrelation coefficient indicates the time series' repeatability and predictability. By using these specific lags as the sampling interval, we can maximize the capture of time series points most relevant for prediction, while effectively discarding data that is irrelevant or exhibits weak dependency. Consequently, this approach facilitates lightweight training and inference while maintaining the precision of time series prediction.

In the top-left section of Fig. 4, the rectangular box represents the input data, while the colored boxes denote rolling series with different offsets. The rolling autocorrelation coefficient, computed using equation (13), determines the clockwork frequencies for the k LSTM subunits by selecting the top k coefficients. The upper-right section of Fig. 4 depicts the LSTM's hidden and memory states, which are divided into k sections, each corresponding to a subunit's cell state. The downward arrow in the first column represents the transmission from long-to short-term dependencies. In network traffic prediction, long-range dependencies and short-range dependencies are merged using multi-layer perception. Specifically, in the proposed adaptive clockwork LSTM model, the memory states and hidden states of the LSTM are partitioned into multiple sub-sections. Each sub-section operates as an independent state, updated at

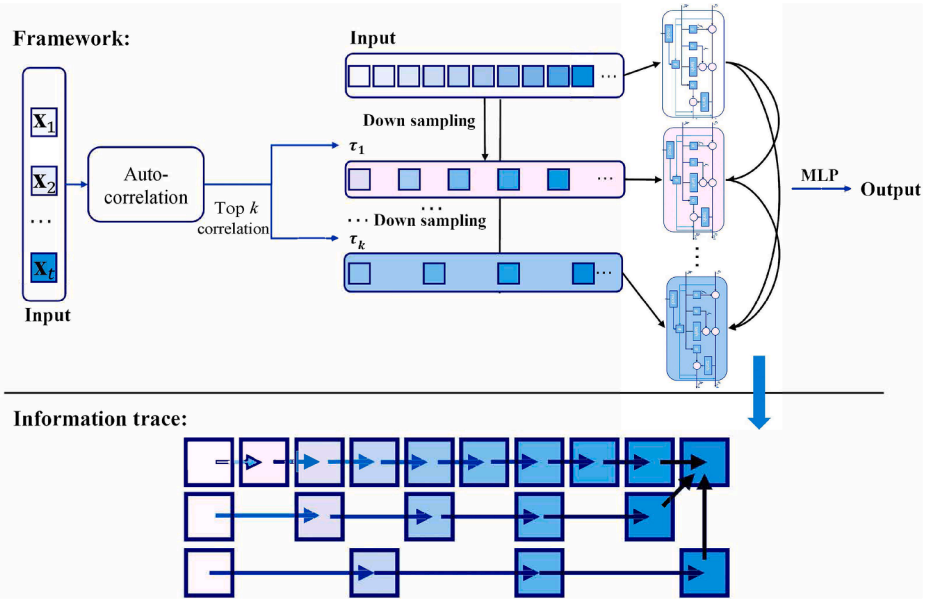


Fig. 4. AC-LSTM framework.

distinct sampling frequencies determined adaptively based on temporal characteristics. This modular design enables the model to efficiently capture temporal dependencies across different time scales. At the final stage, the outputs of these individual sub-states are integrated through a multi-layer perceptron (MLP) to produce a unified prediction. Results are also conducted using the output layer which is the same as the classic LSTM's output layer.

4.4. *Training pipeline of AC-LSTM*

In this section, we outline the detailed training pipeline of AC-LSTM in Algorithm 1. In general, our algorithm consists of the following three stages:

- **Stage 1: sampling rate calculation.** In this stage, we determine the appropriate sampling rate by analyzing the autocorrelation, as described in equation (13). We then select the top K autocorrelations to establish the sampling rates, dividing the hidden state and cell state into multiple segments, each operating at a distinct sampling rate.
- **Stage 2: model forwarding.** During this stage, we forward the model using equations (11) and (7). Temporal data is input into each subunit, which processes the data based on whether the temporal index meets the conditions outlined in equations (11) and (7). Additionally, we generate temporal data, denoted as $\widehat{Y}$.
- **Stage 3: loss computation and backpropagation.** In this final stage, we compute the loss using equation (14) and update the model parameters through the gradient backpropagation mechanism.

Algorithm 1: Training pipeline of AC-LSTM

Require: $X = \{X_1, \cdots, X_m\}$, $Y = \{Y_{m+1}, \cdots, Y_{m+n}\}$ (Input-m-Predict-n)
Ensure: Trained AC-LSTM ϕ parameters
1: Adaptively calculate the sampling rates R_τ using equation (13) and select the top K elements.
2: Divide hidden state h_t and cell state c_t of ϕ into K subunits: h_t^K, c_t^K.
3: **for** $1 \leqslant t \leqslant m$ **do** // Iterate over historical observations
4: **for** $1 \leqslant d \leqslant n$ **do** // Iterate over prediction intervals
5: **for** $1 \leqslant k \leqslant K$ **do** // Iterate over subunits and update states
6: Calculate h_{t+d}^{K-k}, c_{t+d}^{K-k} using equations (11) and (7).
7: **end for**
8: **end for**
9: **end for**
10: Compute $\mathcal{L} \leftarrow \frac{1}{n}\sum_{i=1}^{n} \|\widehat{\mathbf{y}}_{m+i} - \mathbf{y}_{m+i}\|_2^2$.
11: Perform $\nabla_\phi \mathcal{L}$ via backpropagation through time.
12: Update $\phi^* \leftarrow \text{Adam}(\phi, \nabla_\phi \mathcal{L})$.

5. Experiment

5.1. Setup

- **Dataset.** We conducted experiments on two public datasets: the UK Educational Backbone Network dataset [40] and the Conference dataset (CRAWDAD) [41]. UK Educational Backbone Network dataset [40] includes 19,888 sets of network traffic data collected from 9:30 a.m. on November 19, 2004, to 11:11 a.m. on January 27, 2005. The network traffic series were sampled at 5-min intervals. Conference dataset (CRAWDAD) [41] contains 16,329,537 network traffic data points collected from 8:24 a.m. on August 29 to 11:53 a.m. on August 31. We aggregated the dataset items into 1-min intervals using Wireshark and Python, resulting in 3029 samples. The dataset is divided into training, validation, and test sets with a ratio of 8:1:1. A balanced distribution is ensured to facilitate effective training, validation, and testing, as shown in Fig. 5.
  Two distinct network traffic distributions are observed: a double Gaussian distribution, depicted in Fig. 5(a) and (b), and a Pareto distribution, shown in Fig. 5(c) and (d).
  As depicted in Fig. 6, we use auto-correlation analysis to quantitatively illustrate the long-and short-range correlations within the network traffic. This analysis provides a theoretical foundation for using AC-LSTM to improve network traffic prediction. Auto-correlation measures the correlation of a signal with its delayed copy as a function of delay. It is a useful mathematical tool for detecting repeating patterns, such as identifying periodic signals masked by noise or uncovering the fundamental frequency implied by harmonic frequencies.

- **Metrics.** We choose the following evaluation metrics to assess the model's prediction performance:

$$\mathrm{MSE} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2, \tag{14}$$

$$\mathrm{MAE} = \frac{1}{m}\sum_{i=1}^{m}|\hat{y}_i - y_i|, \tag{15}$$

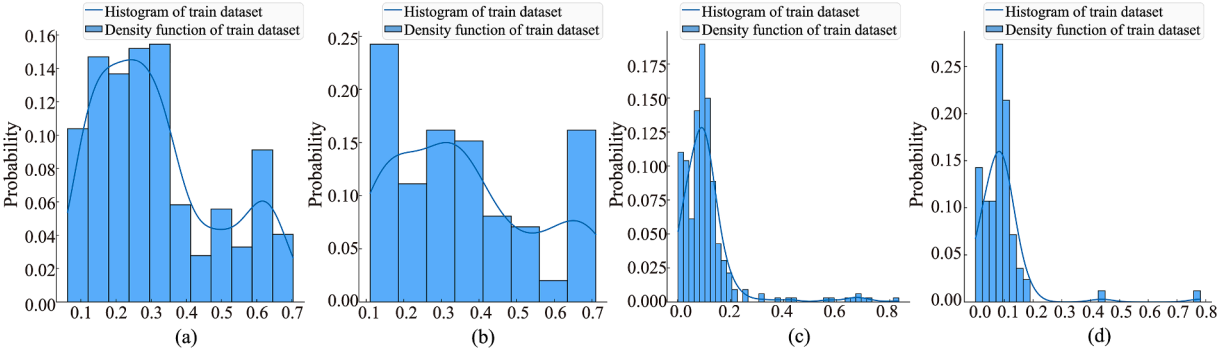


Fig. 5. Dataset distribution overview. (a) Train data [40]. (b) Test data [40]. (c) Train data [41]. (d) Test data [41].

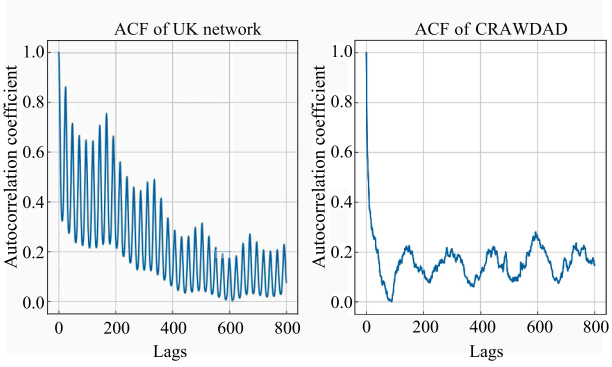


Fig. 6. Dataset plot.

where MSE (Mean Squared Error) evaluates the average squared difference between predicted and actual values, providing an intuitive understanding of the error magnitude. MAE calculates the average absolute difference between predicted and actual values, reflecting the overall error level.

- **Experimental setup.** The experiments were conducted three times on a machine with an Intel Silver 4110 CPU, 64 GB RAM, and NVIDIA GeForce Quadro P2000 GPU. The proposed model was implemented using Torch 1.10 and Python 3.9, trained with the L2 loss function and the ADAM optimizer at an initial learning rate of $10e^{-4}$.
- **Hyper-parameters.** Grid search was used to tune the model's hyperparameters, including the input window length, batch size, number of epochs, number of hidden layers, number of hidden layers, and number of modules. The goal was to identify the optimal settings for the model. MAE was used as a performance metric, with lower MAE indicating better prediction accuracy. The parameter optimization results are shown in Table 2.

5.2. Result analysis

- **Prediction performance analysis.** This experiment evaluates the multi-step prediction performance of several models, including feedforward neural networks (FNN), recurrent neural networks (RNN), long short-term memory (LSTM) networks, AC-LSTM, transformer, autoformer, and Koopa, as shown in Tables 3 and 4. We use MSE and MAE as evaluation metrics, where lower values indicate higher prediction accuracy.
  For the UK educational network dataset, the hourly sampling granularity is adopted, and network traffic is characterized by a strong 24-h cycle. Our model leverages this temporal resolution to successfully capture these daily patterns. **When the history length is 12**, the transformer model achieves lower MSE and MAE than all other models. For instance, with a prediction length of 3, its MSE (0.073) and MAE (0.181) are far superior to the next-best model. This is because when the historical information is insufficient to cover a full 24-h cycle, the Transformer's ability to directly model local and global relationships allows it to surpass other models. **When the history length is 24 or greater**, the AC-LSTM model demonstrates stronger competitiveness. As the history length increases to 24, 36, or 48, AC-LSTM achieves the best performance in most short-to-medium-term prediction scenarios (prediction lengths of 3, 5, and 10). For example, with a prediction length of 5 and a history length of 24, AC-LSTM's MSE of 0.204 and MAE of 0.266 outperform all competing models. This highlights the capability of AC-LSTM in leveraging long-sequence information, particularly the 24-h periodic patterns inherent in this dataset.
  In the CRAWDAD network dataset, when the history and prediction lengths are short, the performance of several models is comparable, with LSTM, RNN, and AC-LSTM alternately achieving the best results. As the prediction and history lengths increase, AC-LSTM demonstrates robust and stable predictive power, achieving the lowest MSE and MAE across all history length settings for a prediction length of 10. This proves that AC-LSTM has a significant advantage in handling long-range dependencies and making long-term predictions. Furthermore, the improvement of AC-LSTM over the baseline LSTM model is highly significant. For instance, on the UK dataset with a prediction length of 3 and a history length of 36, AC-LSTM 's MSE (0.158) and MAE (0.222) are approximately 33.3% and 33.5% lower, respectively, compared to the standard LSTM. This provides strong evidence for the effectiveness of combining auto-correlation and sampling mechanisms in enhancing the accuracy of network traffic forecasting.

- **Effective long-term prediction analysis.** Experiments is conducted on the UK Network dataset to compare the average prediction performance of RNN, LSTM, AC-LSTM, and transformer models across increasing prediction steps (3-step, 5-step, and 10-step), using MAE and MSE as evaluation metrics. The experimental result in Fig. 7 shows that AC-LSTM outperforms the other models in almost all prediction steps. Specifically, for the 3-step prediction, it achieves an MAE of 0.3165 and an MSE of 0.3205, both lower than those of traditional RNN and LSTM, demonstrating higher prediction accuracy. As the prediction step increases, it continues to maintain lower errors. For example, in the 10-step prediction, its MAE and MSE are 0.4935 and 0.3650, respectively, significantly lower than those of RNN. This advantage can be attributed to AC-LSTM 's effective modeling of long-term characters.
- **Sensitive analysis.** The number of candidate sampling frequencies K and the corresponding hidden state dimension are the most influential parameters affecting the performance of AC-LSTM. Therefore, we compared the impact of different values of K (i.e., the number of candidate sampling frequencies) and different hidden state dimensions (hidden dim) on model performance for the prediction task with input-24 and output-24 settings. Fig. 8 illustrates the MAE metric heatmap (lighter colors indicate better performance), the experimental results generally demonstrate that increasing the number of candidate sampling frequencies improves model performance. Similarly, increasing the hidden state dimension tends to enhance performance. However, a larger hidden dimension does not always lead to better results. This may be due to the limited number of samples in the current dataset, which might not be sufficient to support the fitting of models with overly large hidden layers. In a summary, a reasonable configuration of the number of hidden units and candidate sampling frequencies can improve model performance to a certain extent.
- **Efficiency analysis.** In the efficiency analysis, we compared the parameters, training time, and inference time of various models, as shown in Table 5. The proposed AC-LSTM model demonstrates a significant advantage over transformer-based models in terms of both parameter count and inference time. Notably, the AC-LSTM model achieves approximately a 65% reduction in inference time

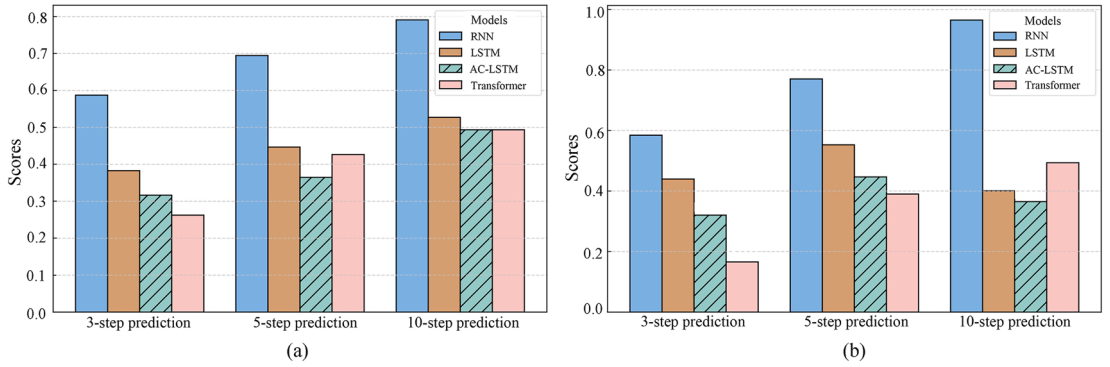


Fig. 7. A comparative prediction step analysis of model performance on the UK network dataset. (a) MAE of model performance comparison. (b) MSE of model performance comparison.

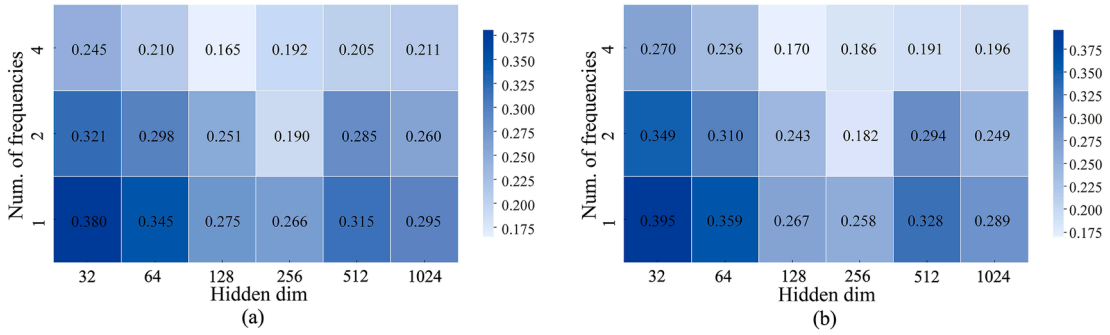


Fig. 8. Sensitive analysis of model performance on the UK network dataset. (a) MAE of model sensitive analysis. (b) MSE of model sensitive analysis.

compared to the average of transformer models. This substantial performance improvement indicates that it is more efficient in handling complex tasks, particularly in real-time applications where enhanced inference speed can greatly improve user experience. It is evident that while the parameter count of the AC-LSTM model (approximately 15,900) is higher than that of traditional RNNs and LSTMs, it remains significantly lower than that of transformer models, such as transformer and autoformer, which exceed 10.5 million parameters. This suggests that the it is designed to be more streamlined, allowing it to maintain high performance while reducing computational resource consumption. Additionally, in terms of training time, it takes 15.6 s, which is slightly higher than some simpler models (like FNN and RNN), but still competitive compared to transformer models. This indicates that it strikes a good balance between training complexity and inference efficiency, making it suitable for applications that require rapid responses.

5.3. Visualization

Fig. 9 presents the prediction results of various time series forecasting models on the UK educational network dataset and the CRAWDAD network dataset. The black curve represents the ground truth, while the other curves correspond to different forecasting models, including RNN, LSTM, AC-LSTM, transformer and Koopa.

From Fig. 9, AC-LSTM exhibits a high degree of alignment with the ground truth in both datasets, outperforming RNN, and LSTM, which show larger deviations. In the UK educational network dataset, RNN and LSTM show significant fluctuations in the CRAWDAD network dataset. These results suggest that traditional neural network models struggle to capture the complexity of network traffic data. In contrast, AC-LSTM provides stable and consistent predictions across both datasets. As shown in Fig. 9(b), AC-LSTM successfully tracks the downward trend in the true data, while transformer models display larger oscillations near the prediction horizon. Overall, AC-LSTM outperforms both traditional neural networks (RNN, LSTM) and certain transformer-based methods (transformer, Koopa) in terms of prediction accuracy and stability, demonstrating AC-LSTM effectiveness in network traffic prediction tasks.

Table 2
Model parameter table.

Input window	Batch size	Epoch	Hidden layers	Layer size	Modules
11	200	120	2	80	7

ARTICLE IN PRESS

Table 3
Prediction errors of UK Educational network dataset. We mark the best MSE and MAE results in bold.

Models		FNN		RNN		LSTM		AC-LSTM		Transformer		Autoformer		Koopa	
Predict	History	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
3	12	1.285	0.945	0.659	0.644	0.458	0.497	0.449	0.476	**0.073**	**0.181**	0.368	0.407	0.226	0.265
	24	0.296	0.346	0.446	0.505	0.152	0.241	**0.144**	**0.232**	0.162	0.271	0.257	0.360	0.323	0.326
	36	0.482	0.483	0.576	0.593	0.237	0.334	**0.158**	**0.222**	0.167	0.251	0.338	0.388	1.331	0.794
	48	0.314	0.366	0.557	0.564	0.365	0.459	**0.304**	**0.317**	0.314	0.411	0.322	0.401	1.197	0.667
5	12	1.575	1.039	1.088	0.862	0.459	0.485	0.451	0.478	**0.221**	**0.343**	0.901	0.698	0.226	0.367
	24	0.385	0.399	0.665	0.623	0.237	0.289	**0.204**	**0.266**	0.283	0.365	0.688	0.645	0.365	0.379
	36	0.495	0.485	0.777	0.685	0.251	0.329	**0.246**	**0.326**	0.366	0.490	0.382	0.428	0.809	0.621
	48	0.397	0.412	0.750	0.663	0.833	0.700	**0.371**	**0.411**	0.418	0.468	0.484	0.502	0.979	0.664
8	12	1.644	1.035	1.271	0.924	0.695	0.654	0.684	0.642	**0.253**	**0.329**	1.477	0.874	0.635	0.636
	24	0.514	0.485	0.802	0.699	0.569	0.542	**0.315**	**0.362**	0.320	0.412	0.636	0.533	0.659	0.559
	36	0.544	0.509	0.973	0.870	0.288	0.366	0.434	0.478	**0.296**	**0.354**	0.833	0.691	0.757	0.595
	48	0.521	0.477	0.931	0.756	0.637	0.480	0.452	0.460	0.482	0.503	0.771	0.653	**0.361**	**0.364**
10	12	1.672	1.043	1.376	0.963	1.006	0.858	1.118	0.855	**0.259**	**0.366**	1.134	0.823	0.749	0.674
	24	0.597	0.540	0.937	0.761	0.329	0.346	**0.314**	**0.336**	0.414	0.483	0.620	0.550	0.508	0.431
	36	0.516	0.484	1.065	0.827	0.375	0.388	**0.351**	**0.385**	0.358	0.412	0.913	0.760	0.576	0.498
	48	0.529	0.488	1.003	0.789	0.552	0.505	**0.521**	**0.486**	0.579	0.575	0.719	0.607	0.612	0.522

ARTICLE IN PRESS

Table 4
Prediction errors of CRAWDAD network dataset. We mark the best MSE and MAE results in bold.

Models		FNN		RNN		LSTM		AC-LSTM		Transformer		Autoformer		Koopa	
Predict	History	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
3	12	0.006	0.056	0.006	0.051	**0.004**	**0.041**	0.006	0.057	0.009	0.078	0.011	0.074	0.005	0.049
	24	0.008	0.063	0.007	0.061	0.008	0.063	**0.007**	**0.056**	0.009	0.078	0.012	0.080	0.011	0.066
	36	0.009	0.067	0.011	0.074	0.012	0.082	**0.008**	**0.066**	0.012	0.087	0.038	0.152	0.047	0.151
	48	0.016	0.087	0.021	0.103	0.023	0.105	**0.011**	**0.077**	0.018	0.138	0.120	0.279	0.128	0.236
5	12	0.011	0.073	**0.009**	**0.064**	0.034	0.130	0.013	0.082	0.015	0.108	0.024	0.109	0.015	0.086
	24	0.013	0.078	0.013	0.081	**0.008**	**0.061**	0.011	0.074	0.014	0.105	0.030	0.124	0.039	0.136
	36	0.015	0.085	0.022	0.108	0.015	0.088	**0.014**	**0.087**	0.014	0.102	0.050	0.165	0.056	0.163
	48	0.016	0.087	0.021	0.103	0.034	0.130	**0.016**	**0.099**	0.017	0.112	0.054	0.176	0.096	0.212
8	12	0.023	0.102	0.021	0.094	**0.017**	**0.084**	0.021	0.097	0.021	0.123	0.040	0.143	0.028	0.109
	24	0.027	0.109	0.029	0.123	0.028	0.128	**0.020**	**0.102**	0.031	0.143	0.058	0.177	0.054	0.158
	36	0.029	0.115	0.037	0.147	0.039	0.146	**0.026**	**0.117**	0.039	0.148	0.106	0.235	0.106	0.227
	48	0.036	0.127	0.057	0.183	0.041	0.192	**0.031**	**0.121**	0.047	0.145	0.132	0.261	0.127	0.247
10	12	0.033	0.122	0.051	0.161	0.043	0.182	**0.029**	**0.112**	0.031	0.147	0.048	0.154	0.037	0.129
	24	0.039	0.132	0.038	0.140	0.045	0.185	**0.029**	**0.129**	0.049	0.158	0.077	0.198	0.079	0.188
	36	0.041	0.137	0.049	0.165	0.059	0.191	**0.039**	**0.146**	0.064	0.182	0.138	0.285	0.109	0.234
	48	0.042	0.139	0.076	0.204	0.076	0.201	**0.042**	**0.145**	0.083	0.219	0.140	0.283	0.103	0.225

Table 5
Model performance comparison.

Model	FNN	RNN	LSTM	GRU	AC-LSTM	Transformer	Autoformer	Koopa
Parameters	1.25×10^2	8.77×10^3	1.84×10^4	1.35×10^4	1.59×10^4	1.05×10^7	1.05×10^7	8.40×10^6
Training time (s)	9.45	1.04×10^1	1.08×10^1	1.22×10^1	1.56×10^1	7.17×10^1	8.22×10^1	5.05×10^1
Inference time (s)	1.00×10^{-5}	7.00×10^{-3}	8.40×10^{-2}	8.70×10^{-2}	9.50×10^{-2}	2.06×10^{-1}	2.89×10^{-1}	3.12×10^{-1}
Num of function calls	21	6.19×10^2	7.27×10^3	5.12×10^3	7.66×10^3	6.76×10^2	7.69×10^2	1.06×10^3

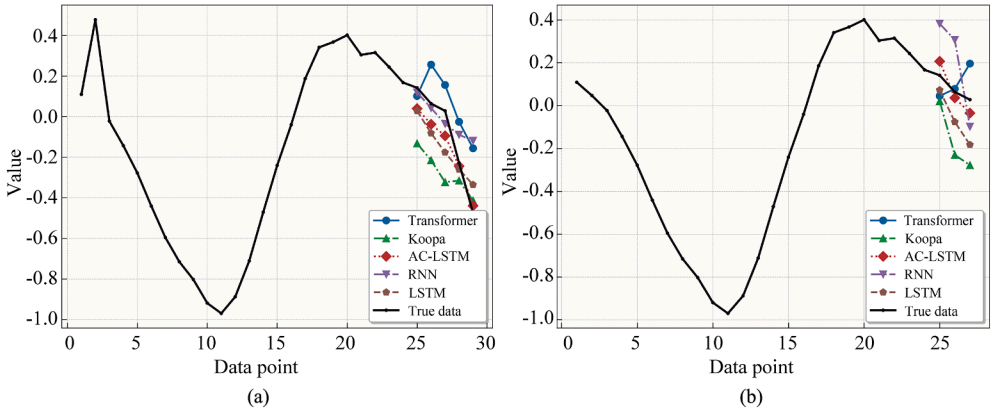


Fig. 9. Comparison of prediction results on different datasets. (a) Prediction results of UK educational network dataset. (b) Predication results of CRAWDAD network dataset.

6. Conclusion

Accurate and efficient network traffic prediction is essential for modern network management, yet challenging due to data complexity and limitations of existing methods like LSTMs for long-range sequences and transformers for high computational cost. To address this, we proposed the adaptive clockwork long short-term memory (AC-LSTM) model, which employs a novel adaptive clockwork mechanism utilizing an adaptive autocorrelation module for dynamic, efficient sampling, thereby effectively capturing both short-term and long-term dependencies. Experimental results demonstrate that AC-LSTM achieves superior performance, significantly improving prediction accuracy while drastically reducing computational time, proving it to be a highly effective and practical solution for efficient network traffic forecasting.

CRediT authorship contribution statement

Yunfeng Ge: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Zonghuan Guo:** Conceptualization. **Peng Wang:** Conceptualization. **Xiaogang Li:** Conceptualization. **Hongyan Li:** Supervision.

Acknowledgments

This work was supported by the Key Program of the National Natural Science Foundation of China (61931017) and General Program of the National Natural Science Foundation of China (62371374).

Declaration of competing interest

There's no financial/personal interest or belief that could affect our objectivity.

References

[1] D. Awduche, A. Chiu, A. Elwalid, I. Widjaja, X. Xiao, Rfc3272: overview and principles of internet traffic engineering. RFC, 2002.
[2] C.Y. Chang, T.G. Ruiz, F. Paolucci, M.A. Jiménez, J. Sacido, C. Papagianni, F. Ubaldi, D. Scano, M. Gharbaoui, A. Giorgetti, et al., Performance isolation for network slices in industry 4.0: the 5growth approach, IEEE Access 9 (2021) 166990–167003.
[3] S. Velrajan, V. Ceronmani Sharmila, Qos-aware service migration in multi-access edge compute using closed-loop adaptive particle swarm optimization algorithm, J. Netw. Syst. Manag. 31 (2023) 17.
[4] Y. Feng, J. Li, J. Mirkovic, C. Wu, C. Wang, H. Ren, J. Xu, Y. Liu, Unmasking the internet: a survey of fine-grained network traffic analysis, IEEE Comm. Surv. Tutorials (2025).

[5] Y. Feng, J. Li, L. Jiao, X. Wu, Towards learning-based, content-agnostic detection of social bot traffic, IEEE Trans. Dependable Secure Comput. 18 (2020) 2149–2163.
[6] Y. Feng, J. Li, D. Sisodia, P. Reiher, On explainable and adaptable detection of distributed denial-of-service traffic, IEEE Trans. Dependable Secure Comput. 21 (2023) 2211–2226.
[7] A. Azzouni, G. Pujolle, A long short-term memory recurrent neural network framework for network traffic matrix prediction, 2017 arXiv preprint arXiv: 1705.05690.
[8] K. Gao, D. Li, L. Chen, J. Geng, F. Gui, Y. Cheng, Y. Gu, Predicting traffic demand matrix by considering inter-flow correlations, in: IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE, 2020, pp. 165–170.
[9] Z. He, C.Y. Chow, J.D. Zhang, Stnn: a spatio-temporal neural network for traffic predictions, IEEE Trans. Intell. Transport. Syst. 22 (2020) 7642–7651.
[10] G. Lai, W.C. Chang, Y. Yang, H. Liu, Modeling long-and short-term temporal patterns with deep neural networks, in: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, 2018, pp. 95–104.
[11] Q. Liu, J. Li, Z. Lu, St-tran: spatial-temporal transformer for cellular traffic prediction, IEEE Commun. Lett. 25 (2021) 3325–3329.
[12] T. Shelatkar, S. Tondale, S. Yadav, S. Ahir, Web traffic time series forecasting using arima and lstm rnn, in: ITM Web of Conferences, EDP Sciences, 2020 03017.
[13] B. Yang, S. Sun, J. Li, X. Lin, Y. Tian, Traffic flow prediction using lstm with feature enhancement, Neurocomputing 332 (2019) 320–327.
[14] W. Zheng, Y. Li, M. Hong, X. Fan, G. Zhao, Flow-by-flow traffic matrix prediction methods: achieving accurate, adaptable, low cost results, Comput. Commun. 194 (2022) 348–360.
[15] B. Feng, Z. Ding, Group: an end-to-end multi-step-ahead workload prediction approach focusing on workload group behavior, in: Proceedings of the ACM Web Conference 2023, 2023, pp. 3098–3108.
[16] S. Huang, Z. Wang, H. Zhang, X. Wang, C. Zhang, W. Wang, One for all: unified workload prediction for dynamic multi-tenant edge cloud platforms, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2023, pp. 788–797.
[17] Y. Li, H. Yuan, Z. Fu, X. Ma, M. Xu, S. Wang, Elastic: edge workload forecasting based on collaborative cloud-edge deep learning, in: Proceedings of the ACM Web Conference 2023, 2023, pp. 3056–3066.
[18] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (1997) 1735–1780.
[19] N. Kitaev, Ł. Kaiser, A. Levskaya, Reformer: the efficient transformer, 2020 arXiv preprint arXiv:2001.04451.
[20] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y.X. Wang, X. Yan, Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting, Adv. Neural Inf. Process. Syst. 32 (2019).
[21] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: long-term forecasting with transformers, 2022 arXiv preprint arXiv: 2211.14730.
[22] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, Adv. Neural Inf. Process. Syst. 34 (2021) 22419–22430.
[23] E.H. Bouzidi, A. Outtagarts, R. Langar, R. Boutaba, Deep q-network and traffic prediction based routing optimization in software defined networks, J. Netw. Comput. Appl. 192 (2021) 103181.
[24] J. Li, H. Wu, Q. He, Y. Zhao, X. Wang, Dynamic qos prediction with intelligent route estimation via inverse reinforcement learning, IEEE Transactions on Services Computing 17 (2023) 509–523.
[25] M. Khasawneh, A. Azab, S. Alrabaee, H. Sakkal, H.H. Bakhit, Convergence of iot and cognitive radio networks: a survey of applications, techniques, and challenges, IEEE Access 11 (2023) 71097–71112.
[26] W. Osamy, A.M. Khedr, A. Salim, A.I. Al Ali, A.A. El-Sawy, A review on recent studies utilizing artificial intelligence methods for solving routing challenges in wireless sensor networks, PeerJ Comput. Sci. 8 (2022) e1089.
[27] J. Wang, K. Zhu, E. Hossain, Green internet of vehicles (iov) in the 6g era: toward sustainable vehicular communications and networking, IEEE Trans. Green Comm. Networking 6 (2021) 391–423.
[28] I. Sutskever, Sequence to Sequence Learning with Neural Networks, 2014 arXiv Preprint arXiv:1409.3215.
[29] E. Coronado, R. Behravesh, T. Subramanya, A. Fernàndez-Fernàndez, M.S. Siddiqui, X. Costa-Pérez, R. Riggio, Zero touch management: a survey of network automation solutions for 5g and 6g networks, IEEE Comm. Surv. Tutorials 24 (2022) 2535–2578.
[30] D. Alekseeva, N. Stepanov, A. Veprev, A. Sharapova, E.S. Lohan, A. Ometov, Comparison of machine learning techniques applied to traffic prediction of real wireless network, IEEE Access 9 (2021) 159495–159514.
[31] Z. Gao, 5g traffic prediction based on deep learning, Comput. Intell. Neurosci. 2022 (2022) 3174530.
[32] D. Bega, M. Gramaglia, M. Fiore, A. Banchs, X. Costa-Perez, Aztec: anticipatory capacity allocation for zero-touch network slicing, in: IEEE INFOCOM 2020-IEEE Conference on Computer Communications, IEEE, 2020, pp. 794–803.
[33] M. Sun, B. He, R. Li, J. Li, X. Zhang, A survey: network feature measurement based on machine learning, Appl. Sci. 13 (2023) 2551.
[34] A. Azzouni, G. Pujolle, Neutm: a neural network-based framework for traffic matrix prediction in sdn, in: NOMS 2018-2018 IEEE/IFIP Network Operations and Management Symposium, IEEE, 2018, pp. 1–5.
[35] Q. Zhuo, Q. Li, H. Yan, Y. Qi, Long short-term memory neural network for network traffic prediction, in: 2017 12th International Conference on Intelligent Systems and Knowledge Engineering (ISKE), IEEE, 2017, pp. 1–6.
[36] R. Fu, Z. Zhang, L. Li, Using lstm and gru neural network methods for traffic flow prediction, in: 2016 31st Youth Academic Annual Conference of Chinese Association of Automation (YAC), IEEE, 2016, pp. 324–328.
[37] J. Koutnik, K. Greff, F. Gomez, J. Schmidhuber, A clockwork rnn, in: International Conference on Machine Learning, PMLR, 2014, pp. 1863–1871.
[38] A. Vaswani, Attention is all you need, Adv. Neural Inf. Process. Syst 30. (2017).
[39] S. Wu, X. Xiao, Q. Ding, P. Zhao, Y. Wei, J. Huang, Adversarial sparse transformer for time series forecasting, Adv. Neural Inf. Process. Syst. 33 (2020) 17105–17115.
[40] P. Cortez, M. Rio, M. Rocha, P. Sousa, Internet traffic forecasting using neural networks, in: The 2006 IEEE International Joint Conference on Neural Network Proceedings, IEEE, 2006, pp. 2635–2642.
[41] D. Kotz, T. Henderson, Crawdad: a community resource for archiving wireless data at dartmouth, IEEE Pervasive Computing 4 (2005) 12–14.



Yunfeng Ge received the B.Eng. degree from Xidian University in 2021. He has been pursuing the Ph.D. degree under the supervision of Prof. Hongyan Li since 2022. His research interests include machine learning, time series analysis, non-stationary network traffic prediction, and network traffic modeling.



Xiaogang Li received the B.Eng. degree from Xidian University in 2022. He has been working toward the Ph.D. degree under the supervision of Prof. Hongyan Li since 2023. His research interests include GPU communication optimization, resource scheduling in distributed training, time-varying graph theory, and satellite Internet.



Zonghuan Guo began the Ph.D. studies at Xidian University in 2025 under the supervision of Prof. Hongyan Li. Her research interests include GPU communication and time-varying graph theory.



Peng Wang received the B.Eng. and Ph.D. degrees from Xidian University, Xi'an, China, in 2017 and 2023, respectively. He is currently a Postdoctoral Research Fellow with the Information Systems Technology and Design Pillar, Singapore University of Technology and Design, where he was also a Visiting Ph.D. Student from May 2021 to November 2022. His research interests include networking algorithms, scheduling, and graph algorithms.



Hongyan Li is a Professor with Xidian University. Her research interests include time-varying graph theory, large-scale time-deterministic networks, and GPU communication.